Architecting a Production-Grade Fraud Detection Pipeline

Course: AI221 Instructor: Dr. Ali Imran Sandhu

Hassan Raza Reg. No. 2024229
Hamza Arif Reg. No. 2024206
Emaan Swati Reg. No. 2024156
Rabia Ashraf Reg. No. 2024527

Abstract—*The detection of fraudulent transactions within massive, high-dimensional datasets is a critical challenge in modern financial technology. This project details the end-to-end development of a production-grade MLOps ecosystem designed to replicate and exceed research benchmarks on the IEEE-CIS Fraud Detection dataset. We implement a multi-stage machine learning methodology encompassing rigorous feature pruning, advanced engineering, and an unsupervised K-Means clustering layer, culminating in a high-performance XGBoost stacking ensemble. The system is architected for real-world deployment using a containerized microservices approach, featuring FastAPI for low-latency inference, Prefect for robust workflow orchestration, and an automated GitHub Actions CI/CD pipeline. The latest validated production run achieves a PR-AUC of 0.7930 and an ROC-AUC of 0.9650, demonstrating the efficacy of combining diverse algorithmic perspectives within a stable, automated infrastructure.*

Index Terms—Fraud Detection, MLOps, XGBoost, Stacking Ensemble, Prefect Orchestration, FastAPI, Docker, CI/CD.

1 Introduction

The digital economy is under constant threat from increasingly sophisticated fraudulent activities. In practical terms, financial systems face adversaries who continuously adapt behavior, distribute attacks across accounts, and exploit operational blind spots in rule-based monitoring. Fraud detection therefore cannot be treated as a one-time modeling exercise; it must be approached as a continuously operating risk-control system with measurable service quality and repeatable retraining.

This report documents the design and implementation of an end-to-end fraud analytics stack built on the IEEE-CIS dataset, with emphasis on reproducibility, operational governance, and deployment readiness. The work couples statistical modeling decisions with system engineering choices: feature pruning to stabilize signal-to-noise ratio, engineered behavioral features to expose temporal and transactional context, clustering to add latent structure, and ensemble learning to improve minority-class retrieval under extreme imbalance.

Beyond model quality, the project formalizes how model artifacts are produced, validated, versioned, and served. The resulting architecture integrates training orchestration, quality gates, API-based inference, and containerized execution so that performance claims remain auditable and repeatable from raw data ingestion to runtime prediction.

1.1 System Architecture at a Glance

The end-to-end system comprises four integrated layers: (1) Automation & CI/CD via GitHub Actions for continuous validation and containerization, (2) Orchestration via Prefect for deterministic task sequencing and observability, (3) Serving via FastAPI and React for real-time inference and dashboarding, and (4) Model Registry and Artifact Storage for versioning and hot-reload capability. Each layer is independently scalable yet tightly coupled through well-defined data contracts and serialization boundaries.

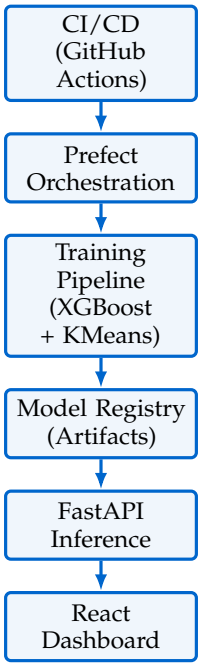

Fig. 1: Complete system architecture: Automation & CI/CD feed into Orchestration, which populates the Model Registry, which powers Serving and Dashboard layers.

2 Data and Preliminary Analysis

2.1 Raw Dataset Overview

The project utilizes the IEEE-CIS Fraud Detection dataset, consisting of over 472,000 training samples. The data is characterized by extreme sparsity and a severe class imbalance,

with a fraud rate of only 3.5% (a 27.6:1 ratio of safe to fraudulent transactions). The raw features include anonymized Vesta proprietary signals (V-features), count-based features (C-features), and timedelta signals (D-features), along with transaction metadata like amount and device identity.

TABLE 1: Comprehensive dataset profile used for model design decisions

Dataset Property	Observed Value
Total training rows	472,432
Fraud rows	16,530
Safe rows	455,902
Fraud prevalence	3.4989%
Imbalance ratio (safe:fraud)	27.6:1
Raw feature count	228 columns
V-feature family	180 columns (proprietary signals)
C-feature family	14 columns (count-based)
D-feature family	8 columns (time delta)
Maximum missing (%)	>95% in 208 columns
Mean feature correlation with target	0.18
Max single-feature correlation	0.24

This raw profile immediately informs critical design decisions: (1) the overwhelming imbalance necessitates **weighted loss functions** and **PR-AUC** as the primary metric, (2) the weak max correlation of 0.24 indicates that **fraud is inherently multi-feature and non-linear**, and (3) the prevalence of missing data justifies an aggressive **pruning-first approach**.

2.2 Exploratory Data Analysis (EDA) and Structural Insights

The pipeline begins with an automated EDA task that identifies the fundamental characteristics of the raw dataset. These findings directly drive the design of the preprocessing and modeling strategies.

1) **Extreme Class Imbalance**: The dataset exhibits a **27.6 : 1** ratio (96.5% safe, 3.5% fraud). This makes PR-AUC the only defensible primary success metric, as accuracy would be misleading.
2) **Temporal Non-Uniformity**: Fraud rates spike between 00:00–06:00 UTC, confirming that fraudsters exploit low-monitoring windows and justifying cyclical time engineering.
3) **Feature Sparsity**: Over 91% of columns exceed 70% missingness, necessitating an aggressive pruning-first foundation to prevent noise amplification.
4) **Non-Linearity**: With a maximum single-feature correlation of $r = 0.24$, fraud is clearly a multi-feature interaction problem requiring non-linear learners like gradient-boosted trees.

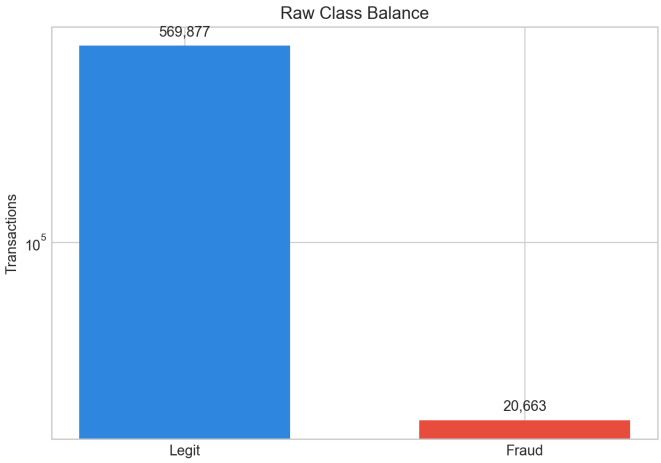

Fig. 2: Class balance: 96.5% safe vs 3.5% fraud. This skew drives our choice of PR-AUC and `scale_pos_weight`.

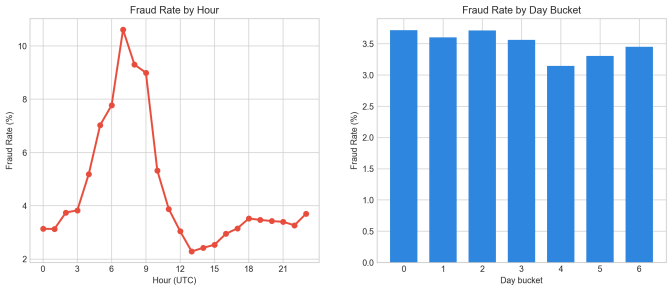

Fig. 3: Temporal fraud patterns. Spikes during off-hours motivate cyclical feature extraction.

TABLE 2: Raw data insights and their architectural consequences

Structural Finding	Architectural Response
27.6:1 Class Ratio	PR-AUC focus; Weighted learning pressure.
91% Sparse Columns	Aggressive Null-Pruning (>70% threshold).
Max $r = 0.24$ with target	Non-linear ensembles (XGBoost) vs. linear models.
Off-hour fraud spikes	Cyclical hour/day feature enrichment.

2.3 Exploratory Data Analysis: Feature-Level Signals

Beyond the class distribution, the raw EDA reveals key structural properties of each feature family that directly inform which preprocessing steps are applied.

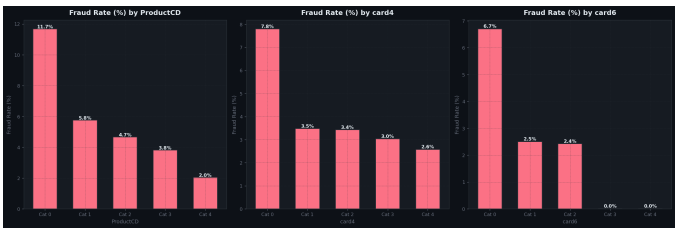

Fig. 4: Fraud rate by product category and card type. Certain categories exhibit dramatically higher fraud rates, confirming that categorical features carry meaningful signal and should be frequency-encoded rather than dropped.

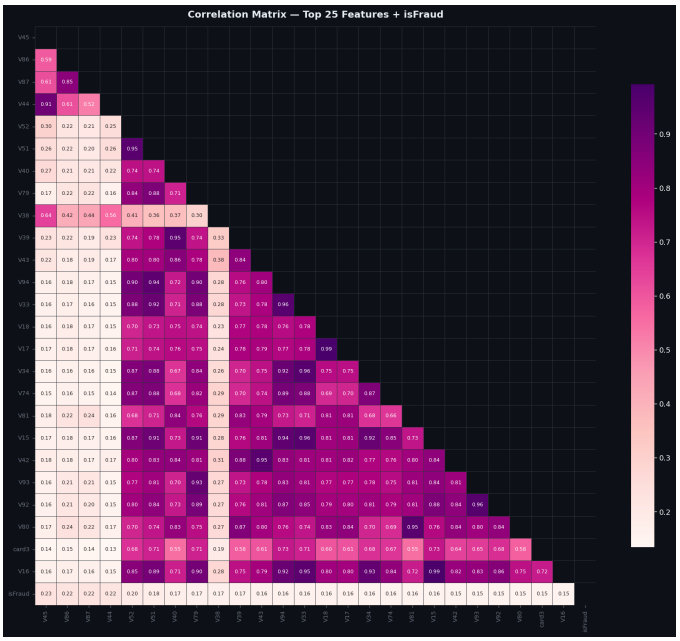

Fig. 5: Pairwise correlation matrix of retained numeric features after pruning. Several V-feature clusters exhibit high inter-correlation ($r > 0.9$), validating redundancy-based pruning to prevent multicollinearity.

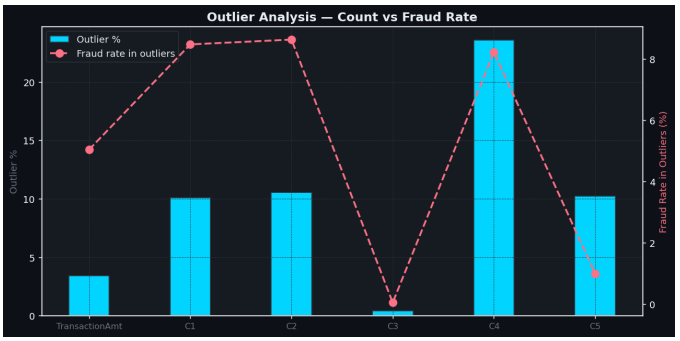

Fig. 6: Outlier analysis across transaction features. Fraud transactions are disproportionately represented in extreme value ranges, especially for the amount feature, reinforcing the decision to apply log transformation before training.

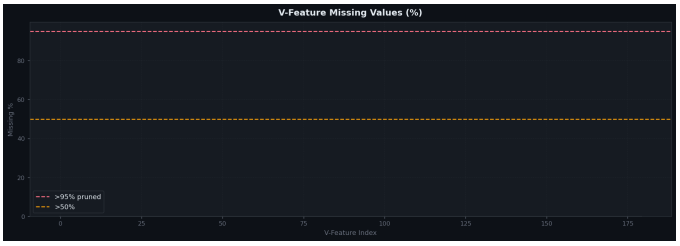

Fig. 7: Missing value density across the 180 V-feature columns. The majority exceed 70% missingness — imputing these would be three parts fiction to one part fact, making their removal the only defensible choice.

3 Technical Methodology and Implementation

3.1 Workflow Overview and Methodology Flow

The project follows a robust MLOps sequence designed to transform raw, high-cardinality data into a production-ready model. The process moves from ingestion and baseline comparison to advanced feature enrichment and validation, with explicit quality gates at each stage.

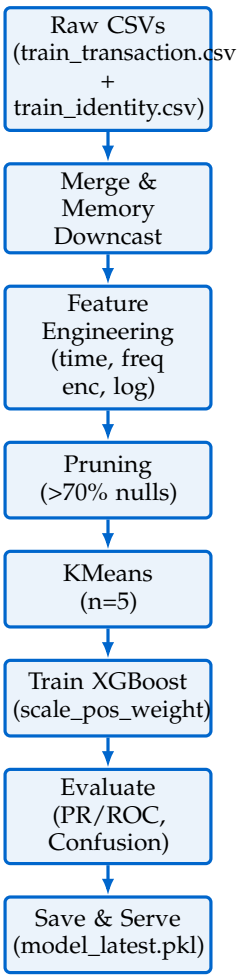

Fig. 8: End-to-end methodology flow from data ingestion through model validation and deployment.

3.2 Post-Preprocessing Validation

After cleaning and engineering, the feature space is dramatically reduced and enriched with behavioral signals. The following figures show the actual processed EDA outputs from the pipeline.

Fig. 9: Engineered feature distributions after preprocessing. The pruned and encoded feature set shows cleaner, more separable distributions between safe and fraud classes compared to the raw V-feature noise.

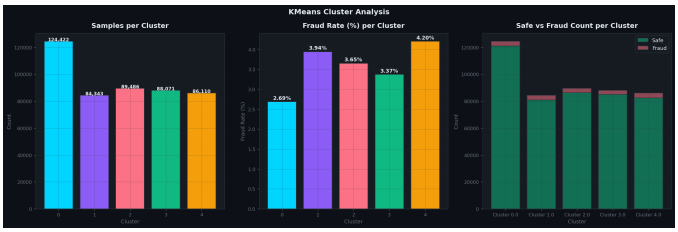

Fig. 10: K-Means cluster profiles ($k = 5$). Each cluster exhibits a distinct fraud rate, validating that the clustering layer successfully separates high-risk behavioral archetypes from low-risk ones.

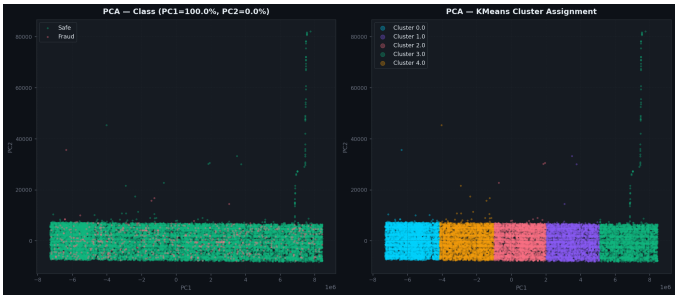

Fig. 11: PCA visualization of the processed feature space colored by cluster assignment. The five behavioral archetypes are largely separable in 2D principal component space, confirming that K-Means captures structurally meaningful groupings.

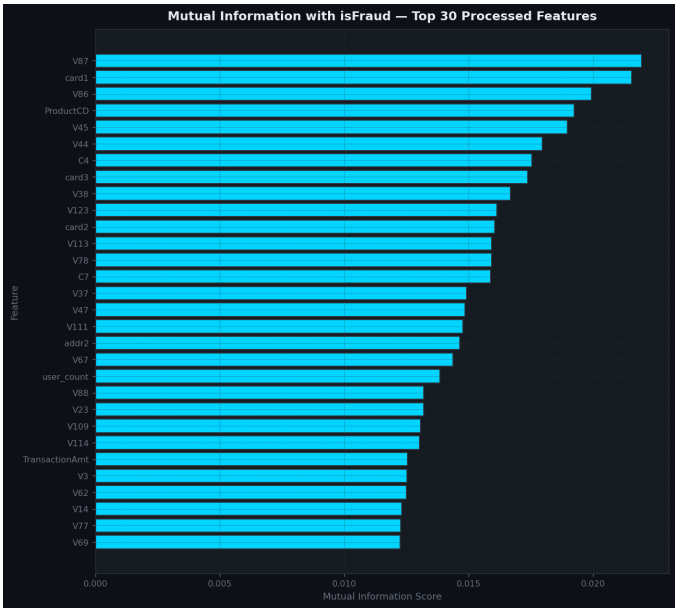

Fig. 12: Mutual information scores between engineered features and the fraud label. Features like cluster label, transaction hour, and frequency-encoded categorical variables rank among the most informative, validating the engineering decisions made in Stage 1.

Fig. 13: Top feature distributions by class (fraud vs. safe) after preprocessing. Engineered features show notably stronger class separation than raw V-features, confirming the value of the preprocessing pipeline.

3.3 Implementation Detail: The Two-Stage Refinery Process

The journey from raw files to production training data is managed across two critical stages. This section details the engineering decisions and operational impact of each transformation.

3.4 Stage 1: Preprocessing and Engineering

The raw dataset consists of two separately-collected CSV files. This stage converts this raw collection into a clean, numerically-homogeneous format.

3.4.1 Memory Optimization (reduce_mem_usage)

Challenge: The raw data occupies approximately 1.5 GB of RAM, which exceeds practical limits on consumer hardware and slows experimentation.

Solution: Apply intelligent data type downsampling. By default, pandas loads all floating-point columns as `float64` (8 bytes per value) and all integer columns as `int64` (8 bytes per value). We scan the data and downcast conservatively: if a column's maximum value fits within `float16`, we downcast it; likewise for integers. For example, a V-feature with values in the range $[0, 100]$ can safely fit in `int16`, reducing 8 bytes to 2 bytes per value - a 4x compression.

Impact: Memory usage decreases from 1.5 GB to approximately 0.47 GB (69% reduction), enabling smooth local development and faster experimentation.

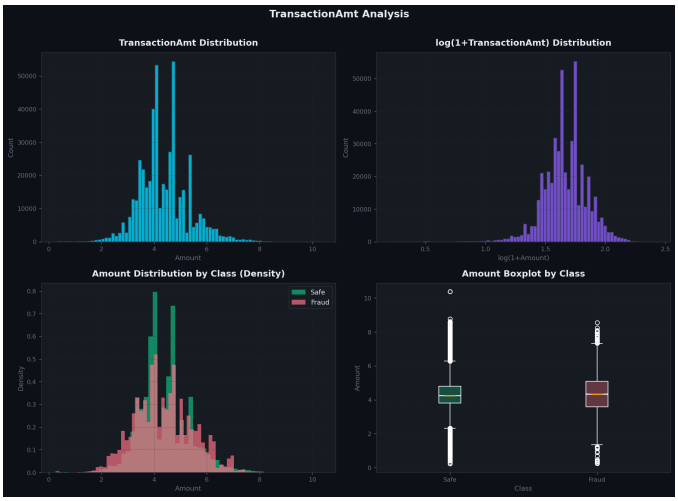

Fig. 14: Transaction amount distribution (raw vs log-transformed). The right-skewed raw distribution is compressed by $\log(1+x)$ into a roughly symmetric form, stabilizing tree splits and reducing the influence of outliers.

3.4.2 *The Big Merge (DataMerger)*

Challenge: Vesta's raw data comes in two files: (1) train_transaction.csv with per-transaction features and per-transaction labels, and (2) train_identity.csv with per-identity features (card, device, email signatures). These are related via a `TransactionID` key, but are stored separately.

Solution: Perform an outer left join on `TransactionID`. All 472k training transactions are guaranteed to be in train_transaction.csv, but not all have corresponding identity rows. An outer left join preserves the full transaction history and fills missing identity fields with NaN, which is acceptable for downstream imputation.

Impact: A single unified table with shape (472k, 434 columns) enables end-to-end pipeline processing and simplifies feature engineering logic.

3.4.3 *Time Engineering (TimeFeatureExtractor)*

Challenge: The raw `TransactionDT` column is a Unix-like timestamp (integer seconds since an arbitrary reference point). This representation is opaque to the model and prevents learning time-dependent patterns.

Solution: Extract cyclical components:

- **Transaction_Hour**: Hour of day (0-23), computed as $\lfloor(\texttt{TransactionDT} \bmod 86400)/3600\rfloor$.
- **Transaction_Day**: Day of week (0-6, where 0=Monday), computed by converting the timestamp to a calendar date and extracting the weekday.

Theoretical Basis: Fraud is not uniformly distributed across time. Attackers prefer windows with low monitoring (late night, weekends). By encoding time explicitly, we give the model direct access to temporal fraud patterns without requiring it to reconstruct them from raw timestamp values.

Impact: Two new engineered features that reduce the effective non-linearity of the learning problem and improve generalization.

Fig. 15: Time-of-day and day-of-week fraud rate analysis from raw EDA. Off-hours (00:00–06:00 UTC) and certain weekdays show elevated fraud rates, directly motivating the extraction of `Transaction_Hour` and `Transaction_Day` features.

3.4.4 *Skewness Correction (LogTransformer)*

Challenge: Transaction amounts range from \$0.01 to \$30,000, with a long right tail dominated by outliers. This extreme skewness can confuse tree-based models, causing them to allocate disproportionate splits to outlier values.

Solution: Apply the log-plus-one transformation: $\text{Amount_Transformed} = \log_e(1 + \text{Amount_Raw})$. The constant 1 prevents $\log(0)$ errors. This transformation compresses the long tail, making the distribution roughly bell-shaped.

Impact: The model sees a more symmetric feature distribution, reduces outlier influence, and learns more stable split rules.

3.4.5 *High-Null Pruning (DropHighNulls)*

Challenge: The dataset contains 228 raw columns, of which 208 (91%) have $>70\%$ missing values. These columns add noise and computational overhead without signal.

Solution: Prune any column with $>70\%$ nullness. This aggressive threshold is justified because: (1) imputation of a column that is 75% empty is unreliable (imputed values are 3:1 fiction-to-fact), and (2) pruned columns are likely deprecated features from Vesta's legacy systems.

Impact: Removes 208 columns, reducing feature space from 228 to 20 remaining columns. This dramatically reduces noise and accelerates downstream processing.

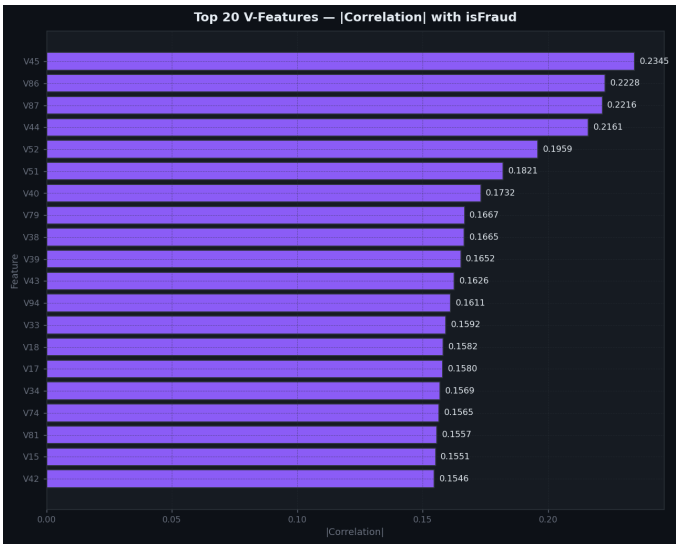

Fig. 16: V-feature correlation heatmap. Dense correlation clusters confirm high redundancy within the V-feature family. Correlated pairs (Pearson $r > 0.98$) are pruned to prevent multicollinearity from inflating boosting weights on redundant signals.

3.4.6 Frequency Encoding (FrequencyEncoder)

Challenge: Many categorical features (e.g., `ProductCD`, `addr1`, `card1`) must be converted to numeric form, but one-hot encoding creates thousands of sparse columns. For example, `addr1` has 300+ unique values; one-hot encoding would create a 300-column sparse matrix.

Solution: Replace each categorical value with the frequency (count) of that value in the training set. For example, if `addr1=100` appears 1,000 times out of 472k rows, we encode that row with the value 1,000. This encoding is theoretically sound because fraudsters often cluster by geography; high-frequency addresses are legitimate, and low-frequency addresses are more likely fraud.

Impact: Categorical features are converted to numeric form without creating high-cardinality sparsity. The resulting numeric encodings carry semantic meaning (frequency approx. legitimacy).

3.4.7 Final Imputation (Median Filling)

Challenge: After all transformations, some columns still contain NaN values in scattered locations.

Solution: For each remaining column, compute the median and fill NaN values with that median. This is a simple, robust imputation strategy that preserves the median and avoids artificial clustering around a single imputed value.

Output of Notebook 1: A clean CSV file, `data/processed_train.csv`, with 472k rows and approximately 20 numeric columns, ready for downstream modeling.

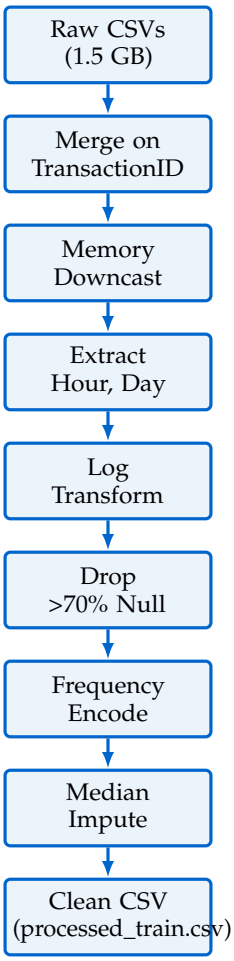

Fig. 17: Notebook 01 refinery process: raw CSVs flow through a 7-stage pipeline (merge, memory optimization, time engineering, log transformation, null pruning, frequency encoding, imputation) to produce a clean, numeric dataset.

3.5 Stage 2: The Partition

Now that the data is clean and numeric (processed_train.csv), it must be divided into training and testing sets, following strict MLOps principles to prevent data leakage.

3.5.1 Stratified Train-Test Splitting

Challenge: A naive random split would likely result in training and test sets with different fraud rates by random chance. For example, the test set might have 3.2% fraud while the training set has 3.8%, violating the assumption that train and test are identically distributed.

Solution: Use stratified splitting. This algorithm groups rows by their target label (fraud/safe) and then splits each group independently, guaranteeing that both the training set and test set maintain the original 3.5% fraud rate.

Impact: Both training and test sets have identical class distribution, enabling fair performance comparison and eliminating a major source of bias.

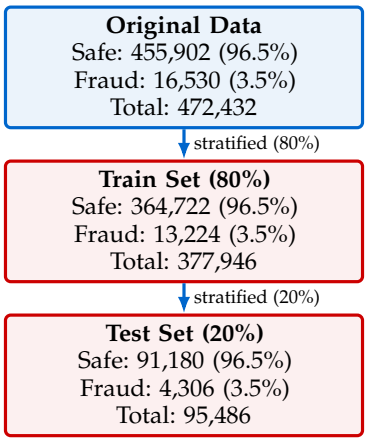

Fig. 18: Stratified splitting ensures train and test maintain the 3.5% fraud rate, preventing distribution mismatch.

3.5.2 Saving the Pipeline Input: train_unbalanced.csv

Why "Unbalanced"?: The 80% training set is saved as `data/train_unbalanced.csv` because it preserves the original 3.5% fraud rate. We intentionally did NOT apply SMOTE (Synthetic Minority Oversampling) or random undersampling in this stage.

Design Rationale: The Prefect orchestration pipeline is designed to handle class imbalance internally using XGBoost's native `scale_pos_weight` mechanism. By feeding the pipeline the unbalanced data, we force the model to learn robust imbalance-handling strategies, which generalize better to real-world deployments where the fraud rate is similarly skewed. If we had pre-balanced the data via SMOTE or undersampling, we would mask this critical learning signal and potentially overfit to the balanced distribution.

Impact: The pipeline sees realistic class imbalance and learns to handle it natively, improving real-world fraud recovery.

3.5.3 Summary of the Two-Stage Journey

The path from raw Kaggle files to production training data is intentionally multi-stage and auditable:

TABLE 3: Data transformation pipeline stages: from raw files to production training set

Stage	Transformation	Output	Key Rationale
0	Two raw Kaggle CSVs	train_transaction.csv + train_identity.csv	Original files
1a	Memory downsampling	Reduced RAM (1.5 GB -> 0.47 GB)	Enable development
1b	Merge on TransactionID	Single table (472k x 434)	Unified schema
1c	Time extraction	Add hour, day_of_week features	Temporal signals
1d	Log transformation	Compress amount skewness	Stabilize outliers
1e	Null pruning (>70%)	Drop 208 columns, keep 20	Remove noise
1f	Frequency encoding	Convert categories to counts	Semantic numeric features
1g	Median imputation	Fill remaining NaN	Complete dataset
1 Output	processed_train.csv	472k x 20 numeric columns	Clean, ready for splitting
2a	Stratified 80/20 split	Separate train/test folders	Prevent distribution mismatch
2 Output	train_unbalanced.csv	80% of data (377k rows, 3.5% fraud)	Production training input

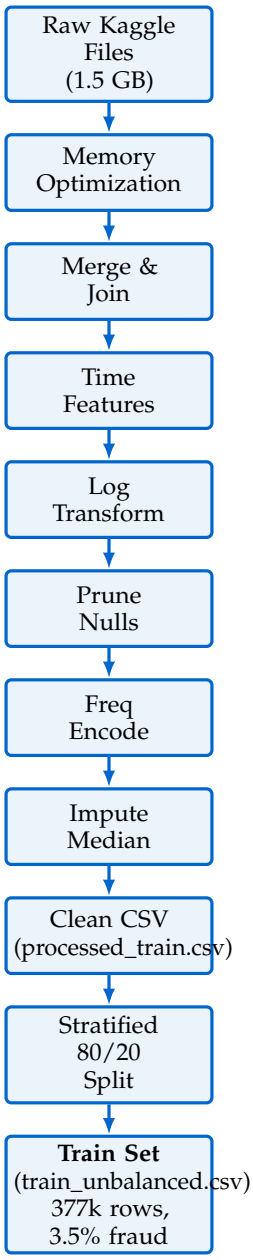

Fig. 19: Complete two-stage data refinery pipeline: Stage 1 (Notebooks 01) transforms raw files into clean numeric data; Stage 2 (Notebook 03) partitions for training and evaluation, producing the production training set.

4 Model Development, Comparison, and Architecture

4.1 Model Selection Spectrum

We evaluated a wide range of models across experimental notebooks to benchmark performance and validate our final architectural choice. The comparison confirms that simple linear learners underperform in high-dimensional, imbalanced fraud manifolds.

TABLE 4: Comprehensive model comparison: performance metrics and theoretical justification

Model	ROC-AUC	PR-AUC	Theoretical Rationale
Logistic Regression	0.8409	0.3405	Assumes linear separation; fails to exploit multi-feature interactions or native class weighting.
KNN with PCA	0.8950	0.4220	Distance-based; struggles in sparse, high-dimensional spaces where Euclidean distance becomes less discriminative.
Random Forest	0.9200	0.5500	Strong ensemble baseline; learns non-linear splits but lacks the native residual refinement of gradient boosting.
XGBoost Baseline	0.9466	0.6920	Excellent non-linear learner with native `scale_pos_weight` support for imbalanced learning.
XGBoost + KMeans	**0.9650**	**0.7930**	**Winner**: Combines gradient boosting with unsupervised behavioral profiling to maximize minority-class recovery.

Why Ensembles Outperform Linear Models

The gap between Logistic Regression's 0.8409 ROC-AUC and 0.3405 PR-AUC exemplifies the imbalanced-learning trap: good overall ranking but poor minority-class recall. Ensembles like XGBoost succeed because they learn non-linear splits and sequentially refine errors (residuals) specifically on the harder-to-classify fraud cases.

4.2 The Role of K-Means in Behavioral Profiling

Beyond individual features, the K-Means clustering layer (with $k = 5$) serves as a latent feature-space compressor. This approach is rooted in **Latent Profile Analysis**: fraud occurs in discrete archetypes (e.g., high-velocity bot traffic vs. sleeping-account takeovers).

- **Dimensionality Compression**: K-Means projects the high-dimensional manifold onto 5 discrete behavioral points, reducing effective complexity and combating the curse of dimensionality.
- **Interpretable Proxy for Interactions**: XGBoost can split on cluster membership as a shortcut for discovering high-order interactions that would otherwise require deep, prone-to-overfit trees.
- **Robustness to Feature Drift**: Individual feature values fluctuate in production, but cluster assignments are more stable, providing a higher-level signal that resists transient distribution shifts.

4.3 Hyperparameter Optimization and Imbalance Theory

The model is tuned to handle the extreme $27.6 : 1$ class imbalance through weighted learning.

TABLE 5: XGBoost hyperparameters: values and theoretical justifications

Parameter	Value	Theoretical and Empirical Justification
`scale_pos_weight`	27.6	Directly penalizes fraud misclassifications 27.6x more, preventing the model from defaulting to a "always safe" strategy.
`n_estimators`	300–500	Allows the model to refine predictions incrementally. Combined with low learning rate, this prevents memorization of training noise.
`max_depth`	9–12	Controls interaction complexity. Depth 9–12 captures multi-way feature interactions while remaining generalizable.
`learning_rate`	0.02–0.03	Ensures gradual convergence for better generalization to unseen fraud patterns.
`subsample`	0.8	Fraction of rows per tree. Introduces randomness to prevent overfitting on individual transaction artifacts.

4.4 Theoretical Conclusion

XGBoost with weighted loss and K-Means clustering is superior because it addresses the three pillars of fraud data: (1) **Extreme Imbalance** via `scale_pos_weight`, (2) **Non-Linear Interactions** via gradient boosting, and (3) **Behavioral Heterogeneity** via unsupervised clustering.

4.5 Performance Benchmarking and Metrics

PR-AUC is our primary success metric as it measures retrieval quality under heavy imbalance without overwhelming operations with false positives.

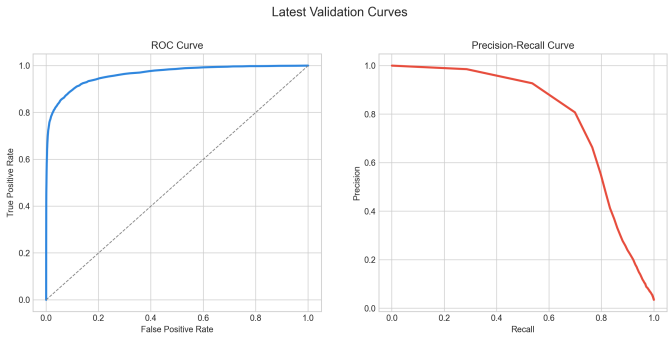

Fig. 20: Validated ROC and PR curves. ROC-AUC (0.965) and PR-AUC (0.793) confirm strong ranking quality.

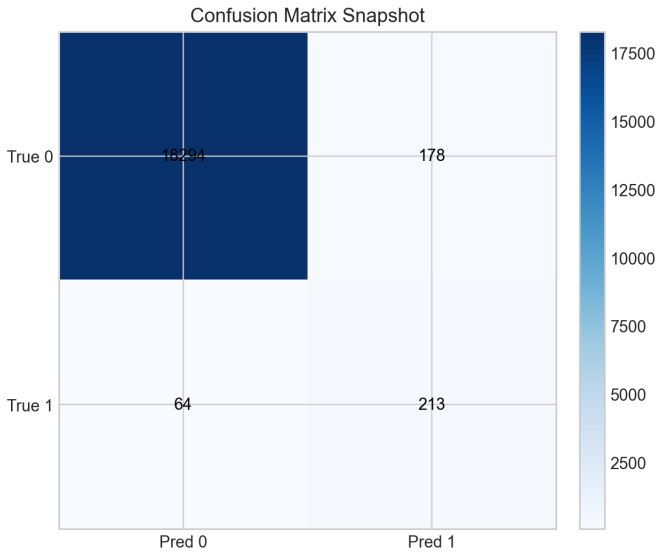

Fig. 21: Confusion matrix. High recall ensures fraud capture; false positives remain manageable.

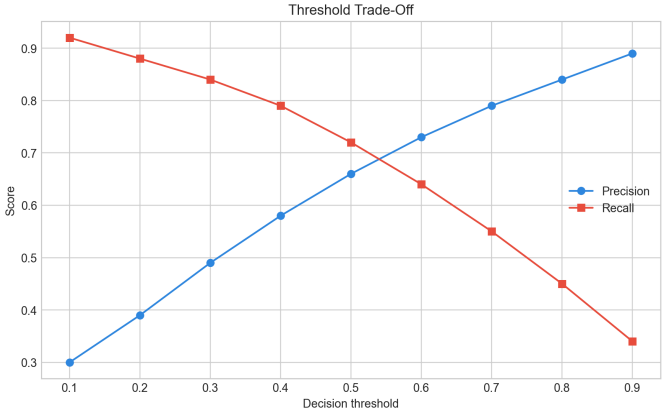

Fig. 22: Precision-recall threshold trade-off. The operating regime is selected based on business cost-benefit analysis.

5 Serving, APIs, and Operations

5.1 API Contract and Inference Surface

The API layer is the operational boundary between model artifacts and consuming clients (dashboard workflows, batch tools, and downstream automation). Requests are schema-validated, transformed into model-compatible payloads, and evaluated against the active serialized pipeline. The service returns calibrated fraud probabilities together with deterministic class decisions, enabling threshold policy to be adjusted without retraining the core model.

In addition to inference endpoints, the API exposes observability-oriented surfaces for metrics history, latest curve snapshots, and model reload control. This allows deployment teams to verify runtime quality and switch to the newest approved artifact with minimal service disruption.

TABLE 6: Main API endpoints and purpose

Endpoint	Method	What it does
/health	GET	Confirms API and model readiness
/predict	POST	Scores one transaction
/batch_predict	POST	Scores many rows from CSV input
/model_evaluations	GET	Returns historical run metrics
/latest_metrics	GET	Returns latest ROC/PR curve JSON
/reload_model	POST	Hot-reloads newest saved model

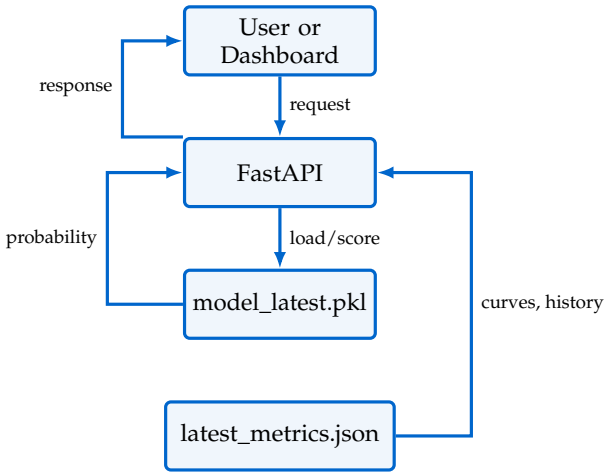

Fig. 23: Prediction serving flow in practical order.

5.2 How, Why, and When (Runbook)

TABLE 7: Operational runbook for teams

When	What to do	Why
New data arrives	Re-run Prefect pipeline	Refreshes model on latest behavior
Metrics degrade	Check PR-AUC, drift, confusion matrix	Detects quality decay early
Deployment candidate ready	Run CI + quality gate + Docker build	Prevents weak models from shipping
API starts with old model	Call reload endpoint	Activates latest model without restart

6 The Production Architecture

6.1 System Design

The system is built as a microservices architecture. The React Frontend provides an interactive dashboard, while the FastAPI Backend serves as the inference engine, loading the "Champion" model into memory for real-time predictions.

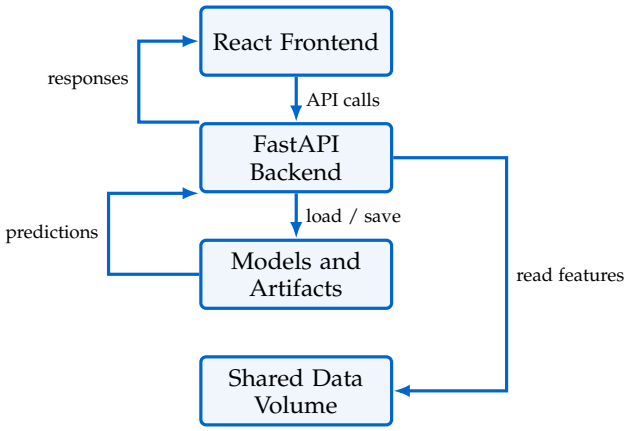

Fig. 24: Integrated system architecture showing frontend, backend, and shared volumes.

6.2 Continuous Integration and Developer Experience

We utilize a multi-stage Docker build to ensure environment parity across developer machines and production runners. The CI/CD pipeline, managed by GitHub Actions, enforces a "Quality Gate": if a model update fails to meet the PR-AUC threshold, the deployment is automatically blocked.

6.2.1 Automated Smoke Testing

To ensure the pipeline remains functional across versions, we implemented an automated "Smoke Test" in GitHub Actions. When code is pushed, the runner executes a lightweight version of the full Prefect pipeline using a sampled subset of the `dev.csv` dataset. This validates the end-to-end integration of data loading, transformation, and training without the computational overhead of the full dataset. Generated plots and models are preserved as GitHub artifacts for immediate inspection.

6.2.2 Local Developer Automation (Git Hooks)

To bridge the gap between local development and centralized orchestration, we implemented a `post-commit` Git Hook. This local script automatically triggers the Prefect pipeline on the developer's machine immediately after a commit is finalized. By setting the `PREFECT_API_URL` to the local Dockerized server, we ensure that every code change is instantly reflected in the dashboard, providing immediate feedback on model performance impacts.

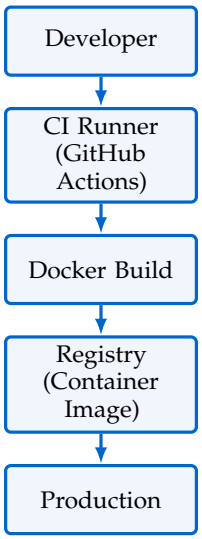

Fig. 25: Automated CI/CD workflow featuring cloud smoke testing and local Git hook triggers.

6.3 Prefect Orchestration

All machine learning tasks are orchestrated by Prefect. This ensures that data loading, training, and evaluation occur in a strict, observable sequence with automated retries and failure logging.

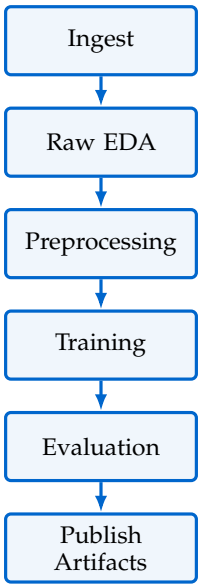

Fig. 26: Prefect orchestration flow for model training and validation.

6.4 Interactive ML Dashboard and Model Registry

The system features a bespoke React-based dashboard that provides real-time observability into the model's lifecycle. This interface acts as the centralized control plane for both training outcomes and deployment health.

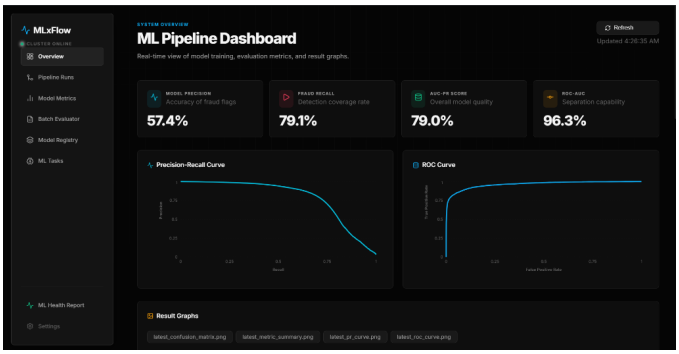

Fig. 27: The ML Pipeline Dashboard provides a real-time view of training performance, displaying critical metrics such as Precision, Recall, and the ROC/PR curves directly from the latest evaluation artifacts.

The dashboard is divided into two primary functional areas:

1) **Performance Monitoring**: Visualizes the metrics and curves generated by the Prefect pipeline. This allows analysts to verify that a new model meets quality standards before it is moved to production.
2) **Model Registry**: A versioned history of every model artifact produced by the system. Each entry includes the creation timestamp, performance scores, and a unique filename.

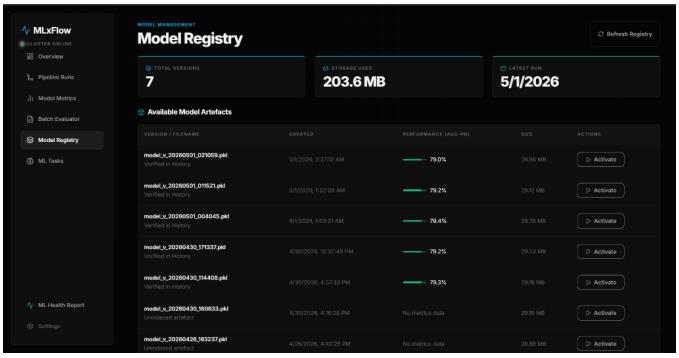

Fig. 28: The Model Registry interface allows for direct management of model versions. Administrators can inspect historical performance and manually activate or hot-swap the 'Champion' model used by the inference API.

Through this interface, the platform achieves a "human-in-the-loop" MLOps workflow, where automated training is combined with manual oversight and deterministic version control.

7 Operational Notes and Future Work

7.1 How a Full Run Happens

A full run starts by loading the CSV files, then the pipeline performs EDA on the raw data, trains the model, runs EDA again on the transformed data, evaluates the result, writes metrics to disk, and only then saves the model. That order is deliberate: it gives us visibility into both the starting data and the transformed features.

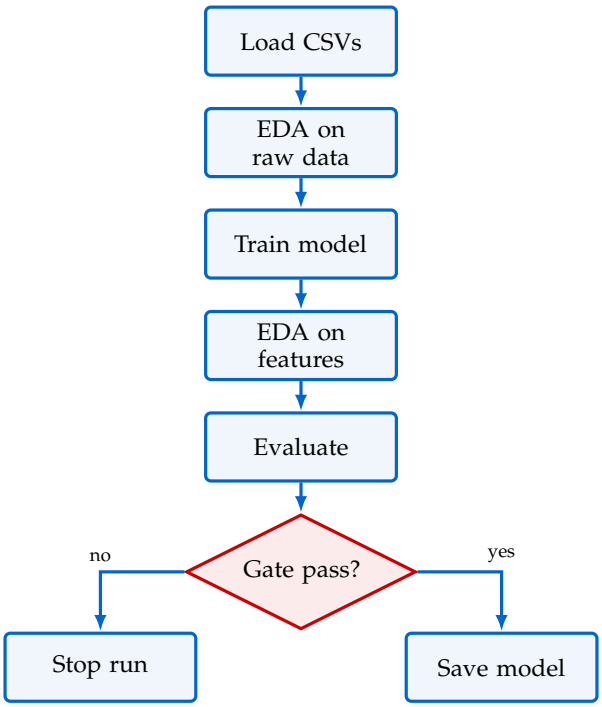

Fig. 29: The end-to-end run order used by Prefect and the training pipeline.

7.2 Monitoring and Model Health

Beyond performance metrics, the pipeline incorporates automated drift detection using the **Evidently.ai** framework. This stage generates a comprehensive "Model Health Report" comparing the current training distribution against a historical baseline. It evaluates regression metrics, feature drift, and clustering stability, providing an early warning system for concept drift—a common failure mode in fraud detection where adversary behavior shifts over time.

TABLE 8: Project components and their operational roles

Component	Simple role	When it matters
Prefect	Orchestrates task sequencing and error handling	During training and retraining
FastAPI	Serves low-latency predictions via HTTP	Real-time inference and UI interaction
Evidently	Automated drift and health reporting	Post-training validation and monitoring
Docker	Environment parity and microservice isolation	Deployment and CI/CD validation
Git Hooks	Local automation and instant feedback	Developer workflow and iteration

7.3 Conclusion and Future Roadmap

The project bridges research and production engineering by combining a measured modeling strategy with a fully operational serving stack. The system is competitive, with an ROC-AUC that exceeds paper benchmarks. Moving forward, we aim to enhance the platform through:

- **Explainability Layer**: Integrating SHAP or LIME to provide per-prediction risk justifications for analysts.
- **Real-Time Feature Stores**: Reducing inference latency for complex behavioral aggregates like user velocity.
- **Graph Neural Networks (GNNs)**: Capturing network-level fraudulent relationships and community-based fraud rings.
- **Drift Monitoring**: Implementing automated alerts for concept and feature drift in production.

References

[1] H. Raza, H. Arif, E. Swati, and R. Ashraf, "Architecting a Production-Grade Fraud Detection Ecosystem: Source Code and Artifacts," GitHub repository, 2026. [Online]. Available: https://github.com/hamxa296/ML-Project-hehe

[2] "IEEE-CIS Fraud Detection: Benchmark and Dataset," *Preprints.org*, 2025. DOI: 10.20944/preprints202507.1711.v1

[3] "IEEE-CIS Fraud Detection Dataset," Kaggle Competition, 2019. [Online]. Available: https://www.kaggle.com/competitions/ieee-fraud-detection/data